Table des matières

Cours 03 — Écrire dans la base : DML et DDL 1
Objectifs 1
Théorie 2
Points clés à retenir 17
Exercices 17

Cours 03 — Écrire dans la base : DML et DDL

Module 3/12 — Formation SQL avancé

Comment lire ce module — Jusqu'ici, vous avez lu des tables que quelqu'un d'autre avait créées et remplies. Ce cours passe de l'autre côté : créer une table, choisir ses types et ses contraintes, y insérer des lignes, les corriger, fusionner une source dans une cible, et savoir à chaque commande si un retour en arrière est encore possible. On reprend la bibliothèque de quartier du Cours 02, cette fois pour la construire. Les deux laboratoires (`Labo_03_OLAP_SQL.md` et `Labo_03_OLTP_SQL.md`) écrivent dans un bac à sable, jamais dans les tables d'AdventureWorks, et c'est sur l'écriture que les deux moteurs divergent le plus.

Où s'arrêter — Ce cours est le plus gros de la formation : environ onze heures, soit quatre à cinq séances de deux heures et demie. La théorie et les deux labos en prennent deux, les exercices et leur corrigé le reste. La coupure tombe après la section 4 : les sections 1 à 4 créent une table et la remplissent, les sections 5 à 9 modifient ce qui existe déjà et disent ce qui s'annule. Les labos se coupent au même endroit, à la fin de leur partie 4.

Objectifs

À la fin de ce module, vous saurez :

- travailler dans un bac à sable, et créer une table de travail à partir d'une requête ;
- créer une table en choisissant ses types, ses valeurs par défaut et sa clé générée, et prévoir ce que le type fait à une valeur insérée ;
- déclarer les cinq contraintes, et distinguer ce qu'un moteur déclare de ce qu'il applique ;
- insérer en nommant les colonnes, depuis une liste de valeurs ou depuis une requête, et récupérer la clé générée ;
- modifier et supprimer sans toucher une ligne de trop, et choisir entre `DELETE` et `TRUNCATE` ;
- fusionner une source dans une table avec `MERGE`, et reconnaître la source en double qui le fait échouer ;
- créer une vue, modifier ou supprimer une structure en sachant ce qui en dépend ;

© 2026 iAlexMG. Tous droits réservés.

- dire, pour chaque commande et sur chaque moteur, si un `ROLLBACK` peut encore la rattraper.

Théorie

La bibliothèque, cette fois en chantier

Les exemples reprennent les trois tables du Cours 02, `membre`, `livre` et `emprunt`, avec les mêmes lignes et la même date du jour, le 20 mai 2026. La différence, c'est qu'on part de rien : les tables n'existent pas encore, et chaque section en construit, en remplit ou en modifie un morceau.

Rappel — Le Cours 01 (section 2) a rangé les commandes SQL en familles. Ce cours travaille les deux qui écrivent : le DDL, qui crée et modifie la structure (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`), et le DML, qui écrit les données (`INSERT`, `UPDATE`, `DELETE`, `MERGE`).

Les résultats et les messages d'erreur cités sont ceux de PostgreSQL 18, dans sa version française. Quand DuckDB, le moteur de l'entrepôt, se comporte autrement, le texte le dit. Snowflake paraît dans les encadrés Portabilité, d'après sa documentation : c'est l'entrepôt infonuagique que les formations Data Engineering et Analytics Engineering vous feront rencontrer.

1. Pourquoi un analyste écrit

Labo : `Labo_03_OLTP_SQL.md` et `Labo_03_OLAP_SQL.md`, partie 1.

Un ingénieur analytique lit beaucoup plus qu'il n'écrit, mais il écrit presque tous les jours. Il garde un résultat intermédiaire dans une table de travail pour ne pas relancer une requête de dix minutes. Il charge le fichier d'un partenaire. Il corrige une valeur mal saisie, publie une vue que d'autres interrogeront, rejoue un chargement qui a échoué pendant la nuit. La suite de la formation le fait souvent : tables de travail au Cours 08, chargements rejouables au Cours 11, schéma de rapport au Cours 12.

La première règle tient en une phrase : on n'écrit pas là où les autres lisent. Chacun travaille dans un **bac à sable**, un espace à lui où une erreur ne casse que son propre travail. Sur les deux moteurs, l'outil naturel est le schéma, un espace de noms à l'intérieur de la base :

```sql
CREATE SCHEMA bac_a_sable;

CREATE TABLE bac_a_sable.essai (x INTEGER);
SELECT * FROM bac_a_sable.essai;
```

Le nom qualifié, `schéma.table`, dit sans ambiguïté où vit la table. Les labos de la formation suivent deux conventions équivalentes : un schéma par cours côté PostgreSQL (`labo03`), un préfixe par cours côté DuckDB (`LABO03_`), pour que les tables de travail restent à côté des tables de `dbo` qu'elles lisent. Dans les deux cas, tout ce qu'un labo crée se retrouve et se supprime d'un coup à la fin.

Qui a le droit de créer un schéma ou une table est, sur un serveur, une affaire de rôles et de privilèges : le compte `postgres` du Cours 00 a tous les droits, et un entrepôt infonuagique comme Snowflake a les siens. DuckDB n'a pas de comptes : qui ouvre le fichier de l'entrepôt peut tout y faire.

© 2026 iAlexMG. Tous droits réservés.

À retenir — Trois habitudes rendent l'écriture sans danger : écrire dans son bac à sable, nommer ce qu'on crée pour pouvoir le retrouver, et compter les lignes qu'une commande va toucher avant de la lancer (section 5).

2. Créer une table

Labo : `Labo_03_OLTP_SQL.md` et `Labo_03_OLAP_SQL.md`, partie 2.

`CREATE TABLE` nomme la table, puis énumère ses colonnes, chacune avec son type et ses options. Voici les trois tables de la bibliothèque, telles qu'on les écrit pour de vrai :

```sql
CREATE TABLE membre (
    membre_id      INTEGER      PRIMARY KEY,
    prenom         VARCHAR(50)  NOT NULL,
    second_prenom  VARCHAR(50),
    nom            VARCHAR(50)  NOT NULL,
    courriel       VARCHAR(100) UNIQUE,
    date_naissance DATE,
    ville          VARCHAR(50)  NOT NULL DEFAULT 'Montréal'
);

CREATE TABLE livre (
    livre_id       INTEGER      PRIMARY KEY,
    isbn           VARCHAR(17)  NOT NULL UNIQUE,
    titre          VARCHAR(200) NOT NULL,
    auteur         VARCHAR(100),
    genre          VARCHAR(30),
    annee_parution INTEGER,
    nb_pages       VARCHAR(10),
    prix           NUMERIC(6,2) CHECK (prix >= 0)
);

CREATE TABLE emprunt (
    emprunt_id   INTEGER      GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    membre_id    INTEGER      NOT NULL REFERENCES membre (membre_id),
    livre_id     INTEGER      NOT NULL REFERENCES livre (livre_id),
    emprunte_le  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    retour_prevu DATE         NOT NULL,
    rendu_le     TIMESTAMP,
    amende       NUMERIC(5,2),
    CHECK (rendu_le IS NULL OR rendu_le >= emprunte_le)
);
```


© 2026 iAlexMG. Tous droits réservés.

Les mots `PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `REFERENCES` et `CHECK` sont des contraintes, et la section 3 leur est consacrée. L'ordre des trois commandes n'est pas libre : `emprunt` fait référence aux deux autres tables, elle se crée donc en dernier. Le reste de cette section s'occupe des types, des valeurs par défaut et de la clé générée.

Le type se choisit d'après ce qu'on fera de la colonne, pas d'après l'allure de ses valeurs (Cours 02, section 4). `isbn` ne contient presque que des chiffres, mais reste du texte : on ne calcule jamais dessus, et un ISBN peut finir par X. `prix` et `amende` sont de l'argent, donc des nombres exacts. `emprunte_le` a besoin de l'heure, `retour_prevu` non. `nb_pages` est du texte pour une mauvaise raison, un vieux fichier où quelqu'un a écrit « inconnu », et la section 7 la corrigera. La longueur d'un `VARCHAR(n)` est un plafond, pas une réservation : un prénom de cinq lettres occupe cinq caractères, pas cinquante. Sur DuckDB, ce n'est même pas un plafond : la longueur s'écrit, puis s'oublie, et le moteur ne la vérifie jamais.

`DEFAULT` donne la valeur qu'une colonne reçoit quand l'`INSERT` ne la nomme pas. Un membre inscrit sans ville est présumé montréalais ; un emprunt enregistré sans heure est daté de l'instant présent, `CURRENT_TIMESTAMP`.

La colonne `emprunt_id` est une **identité** : quand l'`INSERT` ne fournit pas de numéro, le moteur en génère un, en ordre croissant. `GENERATED BY DEFAULT` laisse fournir un numéro explicite, ce qui sert à charger des données existantes ; `GENERATED ALWAYS` le refuse, avec « ne peut pas insérer une valeur pas par défaut dans la colonne ». La section 4 montre le piège que cache la première forme.

Le type protège la colonne, mais pas toujours de la façon qu'on imagine. Il refuse certaines valeurs et en transforme d'autres sans rien dire :

Ce qu'on insère	Dans	PostgreSQL	DuckDB
12,345	`prix NUMERIC(6,2)`	accepté, rangé 12,35	idem
9 999,994	`prix NUMERIC(6,2)`	accepté, rangé 9 999,99	idem
9 999,995	`prix NUMERIC(6,2)`	refusé : arrondi à 10 000,00, il a un chiffre de trop	refusé
un ISBN précédé d'une espace	`isbn VARCHAR(17)`	refusé : « valeur trop longue pour le type character varying(17) »	accepté en entier, 18 caractères
`'2020'`, entre apostrophes	`annee_parution INTEGER`	accepté, converti en 2020	accepté
`'deux mille'`	`annee_parution INTEGER`	refusé : « syntaxe en entrée invalide pour le type integer »	refusé
`'2026-02-30'`	`date_naissance DATE`	refusé : « valeur du champ date/time en dehors des limites »	refusé
40 000	une colonne `SMALLINT`	refusé : « smallint en dehors des limites »	refusé : `SMALLINT` y est un `INT16`

© 2026 iAlexMG. Tous droits réservés.

Portabilité — Snowflake lit autrement deux lignes de ce tableau. D'après sa documentation, il vérifie la longueur d'un `VARCHAR`, comme PostgreSQL, mais fait de `SMALLINT`, `INT` et `BIGINT` trois noms du même type, `NUMBER(38,0)` : le 40 000 y entre sans erreur. Le nom d'un type ne dit pas ce qu'il garantit ; le moteur, si.

Piège — L'arrondi ne prévient pas. `NUMERIC(6,2)` refuse un prix trop grand avec fracas, mais range 12,345 en 12,35 sans un mot. Si la troisième décimale compte, pour un taux de change ou un prix unitaire au dixième de cent, c'est le type qui doit l'accueillir, `NUMERIC(10,4)` par exemple ; l'arrondi se fait ensuite, au moment d'afficher ou de facturer, par un `ROUND` écrit exprès.

Une table peut aussi naître d'une requête. `CREATE TABLE … AS`, qu'on abrège souvent en CTAS, crée la table et la remplit d'un coup :

```sql
CREATE TABLE emprunt_t1 AS
SELECT emprunt_id, membre_id, livre_id, emprunte_le
FROM emprunt
WHERE emprunte_le >= '2026-01-01' AND emprunte_le < '2026-04-01';
```

PostgreSQL répond `SELECT 7` : la table existe et contient les sept emprunts du premier trimestre. C'est la façon la plus rapide de fabriquer une table de travail. Pour n'en garder que la structure, on termine par `WITH NO DATA` sur PostgreSQL, ou par une condition toujours fausse, `WHERE 1 = 0`, qui marche partout.

Piège — Une copie n'hérite pas des garanties. `CREATE TABLE … AS` recopie les noms et les types des colonnes, et rien d'autre : ni clé primaire, ni `NOT NULL`, ni valeur par défaut, ni identité. Une table de travail s'en passe très bien. Une table appelée à durer se crée avec `CREATE TABLE`, puis se remplit par un `INSERT … SELECT` (section 4).

Portabilité — `GENERATED … AS IDENTITY` est la forme de la norme, et PostgreSQL l'applique, mesuré ; `SERIAL`, qu'on lit dans du code PostgreSQL plus ancien, et `AUTOINCREMENT`, que Snowflake pose sur la colonne d'après sa documentation, sont des extensions, chaque moteur ayant la sienne. DuckDB, lui, refuse la forme normalisée, `Constraint not implemented!`, et n'a pas de colonne d'identité du tout : le compteur y est une séquence, créée à part par `CREATE SEQUENCE`, que la valeur par défaut appelle, `DEFAULT NEXTVAL('…')`. Nos deux terrains divergent aussi sur le remplacement d'une table : DuckDB connaît `CREATE OR REPLACE TABLE`, qui supprime et recrée d'un coup, alors que PostgreSQL écrit `DROP TABLE IF EXISTS` puis `CREATE TABLE`.

3. Les contraintes

Labo : `Labo_03_OLTP_SQL.md` et `Labo_03_OLAP_SQL.md`, partie 3.

Une **contrainte** est une règle que la table impose à chacune de ses lignes. Il en existe cinq :

Contrainte	Ce qu'elle interdit	Dans la bibliothèque
`NOT NULL`	l'absence de valeur	un membre sans prénom
`PRIMARY KEY`	deux lignes avec la même clé, ou une clé absente	deux membres numéro 1
`UNIQUE`	deux lignes avec la même valeur, l'absence restant permise	deux livres au même ISBN

© 2026 iAlexMG. Tous droits réservés.

Contrainte	Ce qu'elle interdit	Dans la bibliothèque
`REFERENCES`, la clé étrangère	une valeur absente de la table visée	un emprunt au nom du membre 99
`CHECK`	une ligne qui rend la condition fausse	un prix négatif, un livre rendu avant d'être emprunté

Sur PostgreSQL, chaque violation arrête la commande avec un message qui nomme la contrainte :

```sql
INSERT INTO membre (membre_id, prenom, nom) VALUES (1, 'Nadia', 'Ouellet');
-- la valeur d'une clé dupliquée rompt la contrainte unique « membre_pkey »

INSERT INTO membre (membre_id, prenom, nom) VALUES (7, NULL, 'Ouellet');
-- une valeur NULL viole la contrainte NOT NULL de la colonne « prenom » dans la relation « membre »

INSERT INTO emprunt (emprunt_id, membre_id, livre_id, retour_prevu) VALUES (13, 99, 1, '2026-06-10');
-- une instruction insert ou update sur la table « emprunt » viole la contrainte de clé étrangère
-- « emprunt_membre_id_fkey »

INSERT INTO livre (livre_id, isbn, titre, prix) VALUES (9, '978-2-9800001-9-8', 'Essai', -5);
-- la nouvelle ligne de la relation « livre » viole la contrainte de vérification « livre_prix_check »
```

Le moteur a nommé chaque contrainte à votre place, `membre_pkey`, `livre_prix_check`. On peut les nommer soi-même, `CONSTRAINT prix_positif CHECK (prix >= 0)`, et le message dit alors ce qui cloche dans les mots de l'équipe. Une commande qui viole une contrainte n'écrit rien du tout : un `INSERT` de dix lignes dont une seule est fautive n'en insère aucune.

La clé étrangère protège les deux tables. Elle refuse un emprunt au nom d'un membre inexistant, et elle refuse aussi de supprimer un membre qui a des emprunts : `DELETE FROM membre WHERE membre_id = 5` échoue, parce que Chloé a emprunté le livre 5. On peut demander un autre comportement à la déclaration, `ON DELETE CASCADE` pour supprimer les emprunts avec le membre, `ON DELETE SET NULL` pour les détacher quand la colonne accepte `NULL`. Une bibliothèque se garde bien du premier : supprimer un membre effacerait son historique.

Deux contraintes laissent passer `NULL`, et c'est voulu. `UNIQUE` accepte plusieurs membres sans courriel, parce qu'une absence n'est égale à aucune autre. `CHECK` rejette ce qui est faux, et une condition sur `NULL` n'est ni vraie ni fausse : un livre sans prix passe `CHECK (prix >= 0)` sans difficulté. Si l'absence est interdite, c'est `NOT NULL` qui doit le dire.

`UNIQUE` compare les valeurs telles quelles, avec les mêmes règles que le `=` du Cours 02. Le courriel d'Émile est rangé en majuscules, `EMILE.GAGNON@COURRIEL.TEST` : un second membre inscrit avec `emile.gagnon@courriel.test` passe la contrainte. Sur PostgreSQL, l'unicité se pose alors sur la forme nettoyée, par un index unique, un objet que le Cours 10 présente en détail :

```sql
CREATE UNIQUE INDEX membre_courriel_unique ON membre (LOWER(TRIM(courriel)));
```

Une contrainte peut enfin s'ajouter à une table qui existe déjà, et le moteur vérifie alors les lignes en place avant d'accepter la règle :

© 2026 iAlexMG. Tous droits réservés.

```
ALTER TABLE livre ADD CONSTRAINT nb_pages_en_chiffres CHECK (nb_pages ~ '^[0-9]+$');
-- la contrainte de vérification « nb_pages_en_chiffres » de la relation « livre » est violée par une ligne
```

Le livre au nombre de pages « inconnu » bloque la règle. Une contrainte se pose sur des données propres ; on nettoie d'abord (section 7), on contraint ensuite.

DuckDB applique les cinq contraintes, comme PostgreSQL, et ses messages nomment la colonne et la valeur fautives : `Duplicate key "membre_id: 1" violates primary key constraint`. Il a une limite à connaître, que sa documentation décrit et que le labo OLAP fait toucher du doigt : une modification qui porte sur une colonne indexée, celle d'une clé ou d'une contrainte `UNIQUE`, s'y exécute comme une suppression suivie d'une insertion, et une clé étrangère qui vise la ligne la refuse donc.

Tous les moteurs ne font pas ce travail. D'après sa documentation, Snowflake accepte qu'on déclare une clé primaire, une contrainte `UNIQUE` ou une clé étrangère sur une table ordinaire, et l'enregistre sans l'appliquer : un doublon de clé primaire y entre sans la moindre erreur, et seules `NOT NULL` et `CHECK` sont vérifiées. D'autres entrepôts infonuagiques font le même choix, et c'est dans la documentation du moteur qu'on vérifie, contrainte par contrainte, laquelle est appliquée. Vérifier l'unicité à chaque écriture coûte cher quand on charge des millions de lignes d'un coup ; ces entrepôts préfèrent garder la déclaration comme documentation et laisser le contrôle à ceux qui chargent. L'entrepôt de la formation est dans un cas voisin pour une autre raison : le script du Cours 00 n'y déclare aucune clé. Le contrôle s'écrit en une requête :

```
SELECT membre_id, COUNT(*) AS lignes
FROM membre
GROUP BY membre_id
HAVING COUNT(*) > 1;
```

Aucune ligne : la clé est unique. La formation Analytics Engineering fait de cette requête un test automatique, lancé après chaque chargement.

À retenir — Une contrainte appliquée est une garantie ; une contrainte déclarée mais non appliquée n'est qu'une documentation. Avant de vous fier à une clé primaire, demandez-vous lequel des deux votre moteur vous offre.

Portabilité — Snowflake nuance sa règle de deux façons, d'après sa documentation. Ses tables hybrides, un type de table à part, appliquent la clé primaire, qu'elles exigent, la contrainte `UNIQUE` et la clé étrangère. Et l'attribut `RELY`, posé sur une contrainte d'une table ordinaire, autorise l'optimiseur à s'y fier, par exemple pour retirer une jointure inutile. Il n'est pas posé par défaut, et la documentation laisse à l'équipe la charge de garantir que les données respectent la contrainte : déclarée fiable à tort, elle expose à des résultats inattendus.

4. `INSERT`

Labo : `Labo_03_OLTP_SQL.md` et `Labo_03_OLAP_SQL.md`, partie 4.

`INSERT` ajoute des lignes. La forme de référence nomme les colonnes, puis donne une ou plusieurs lignes de valeurs, dans le même ordre :

```
INSERT INTO membre (membre_id, prenom, second_prenom, nom, courriel, date_naissance, ville)
VALUES (1, 'Sophie', NULL,   'Tremblay', ' sophie.tremblay@courriel.test', '1988-03-14', 'Montréal'),
       (2, 'Émile',  'Jean', 'Gagnon',   'EMILE.GAGNON@COURRIEL.TEST',    '2001-12-31', 'Québec');
```


© 2026 iAlexMG. Tous droits réservés.

PostgreSQL répond `INSERT 0 2`. Le zéro est un vestige historique ; seul le second nombre compte, et c'est lui qu'on vérifie. Les huit livres et les douze emprunts du Cours 02 se chargent de la même façon, les emprunts sans leur colonne `amende`, que la section 5 calculera.

Une colonne que l'`INSERT` ne nomme pas reçoit sa valeur par défaut, ou `NULL` si elle n'en a pas :

```
INSERT INTO membre (membre_id, prenom, nom)
VALUES (7, 'Nadia', 'Ouellet');
```

Nadia n'a ni courriel ni date de naissance, et elle habite Montréal, par défaut. La valeur par défaut ne s'applique qu'à une colonne omise : écrire `NULL` dans la colonne `ville` ne la déclenche pas, et la contrainte `NOT NULL` refuse la ligne. Pour demander la valeur par défaut dans une liste de valeurs, on écrit le mot `DEFAULT` à sa place.

`INSERT … SELECT` remplace la liste de valeurs par une requête, et c'est la forme qui sert le plus en analytique. On crée la table avec ses contraintes, puis on la remplit :

```
CREATE TABLE emprunt_archive (
    emprunt_id INTEGER   PRIMARY KEY,
    membre_id  INTEGER   NOT NULL,
    livre_id   INTEGER   NOT NULL,
    rendu_le   TIMESTAMP NOT NULL
);

INSERT INTO emprunt_archive (emprunt_id, membre_id, livre_id, rendu_le)
SELECT emprunt_id, membre_id, livre_id, rendu_le
FROM emprunt
WHERE rendu_le IS NOT NULL;
```

`INSERT 0 9` : les neuf emprunts rendus sont archivés, et cette fois la table a sa clé primaire.

La liste de colonnes est facultative, et c'est un piège. Sans elle, le moteur associe les valeurs aux colonnes par position, jamais par nom. Retirez-la de l'exemple précédent et intervertissez, par mégarde, deux colonnes du `SELECT` :

```
INSERT INTO emprunt_archive
SELECT emprunt_id, livre_id, membre_id, rendu_le
FROM emprunt
WHERE rendu_le IS NOT NULL;
```

PostgreSQL répond encore `INSERT 0 9`. Les deux colonnes sont des entiers, rien ne l'arrête, et l'archive dit maintenant que l'emprunt 2 a été fait par le membre 4, qui n'y est pour rien. PostgreSQL accepte même une liste plus courte que la table et complète les colonnes manquantes par leurs valeurs par défaut : un `INSERT` sans liste écrit avant l'ajout d'une colonne continue de passer après, sans remplir la nouvelle. DuckDB, lui, refuse une liste de valeurs qui ne couvre pas toutes les colonnes (`table … has 4 columns but 2 values were supplied`), et offre une troisième écriture, `INSERT INTO … BY NAME SELECT …`, qui range chaque colonne dans celle de même nom.

© 2026 iAlexMG. Tous droits réservés.

À retenir — Nommez toujours les colonnes d'un `INSERT`. La liste coûte une ligne, et elle rend la commande insensible à l'ordre des colonnes, à leur nombre et aux changements de structure.

La clé générée, enfin. Pour savoir quel numéro le moteur a donné, PostgreSQL offre `RETURNING`, qui rend des colonnes des lignes écrites. Enregistrons un nouvel emprunt, sans fournir de numéro :

```
INSERT INTO emprunt (membre_id, livre_id, emprunte_le, retour_prevu)
VALUES (3, 2, '2026-05-20 11:00', '2026-06-10')
RETURNING emprunt_id;
```

Erreur : « la valeur d'une clé dupliquée rompt la contrainte unique « emprunt_pkey » », et le détail précise que la clé 1 existe déjà. Les douze emprunts ont été chargés avec leurs numéros, et l'identité, qui ne regarde pas les numéros fournis à la main, repart de 1. Relancez : même erreur, avec la clé 2. Chaque tentative ratée consomme un numéro, et l'annulation ne le rend pas. Le correctif remet le compteur au-delà du plus grand numéro, et ce numéro se lit dans la table plutôt que de se recopier :

```
SELECT setval(pg_get_serial_sequence('emprunt', 'emprunt_id'), MAX(emprunt_id))
FROM emprunt;
```

`pg_get_serial_sequence` retrouve la séquence que la colonne d'identité cache, et `setval` y inscrit le dernier numéro tiré, ici 12, le plus grand chargé. Le même `INSERT` passe alors et rend 13. `ALTER TABLE emprunt ALTER COLUMN emprunt_id RESTART WITH 13` ferait le même travail, mais avec un nombre tapé à la main, qui sera faux au chargement suivant ; la forme calculée se rejoue telle quelle. Sur une table vide, `MAX` rend `NULL`, et `setval` ne fait alors rien, vérifié.

Piège — Après tout chargement avec des numéros explicites dans une colonne d'identité, remettez le compteur au-delà du plus grand numéro. La séquence de DuckDB est tout aussi aveugle, et elle ne se recale pas : `ALTER SEQUENCE` n'y est pas encore pris en charge, et il faut la recréer (labo OLAP, partie 4). Sur un entrepôt qui n'applique pas la clé primaire, le même compteur aveugle ne lève aucune erreur : il crée en silence un second emprunt numéro 1.

Portabilité — `RETURNING` est une extension, absente de la norme, que nos deux terrains ont, mesuré : l'`INSERT` y rend la ligne écrite, numéro compris. La syntaxe de l'`INSERT` de Snowflake ne la porte pas, d'après sa documentation, et c'est le cas à prévoir partout où le moteur d'arrivée ne l'a pas : on relit la table après l'écriture, ou on tire le numéro d'une séquence avant d'insérer, ce qui a l'avantage de le connaître avant même l'`INSERT`.

5. `UPDATE` et `DELETE`

Labo :** `Labo_03_OLTP_SQL.md` **et** `Labo_03_OLAP_SQL.md`**, partie 5.

`UPDATE` modifie des lignes existantes : `SET` dit quoi changer, `WHERE` dit où. La colonne `amende` que lisait le Cours 02 a été remplie ainsi, à 0,25 $ par jour de retard :

```
UPDATE emprunt
SET amende = GREATEST(CAST(rendu_le AS DATE) - retour_prevu, 0) * 0.25
WHERE rendu_le IS NOT NULL;
```


© 2026 iAlexMG. Tous droits réservés.

`UPDATE 9`. Les emprunts 2, 4 et 9 reçoivent 1,00 $, 3,25 $ et 1,75 $; les six autres livres rendus, 0,00 $, grâce au `GREATEST` qui ramène à zéro un retour en avance ; les trois emprunts en cours gardent une amende `NULL`, parce que le `WHERE` les a écartés. DuckDB fait la même soustraction de dates ; d'autres moteurs, comme Snowflake, passent par `DATEDIFF` (Cours 02, section 6). Un `SET` peut changer plusieurs colonnes à la fois, séparées par des virgules.

Le compte rendu, `UPDATE 9`, est la ligne la plus importante de l'écran. Voyez-le à l'œuvre sur une hausse de prix destinée au seul livre 6 :

```
SELECT COUNT(*) FROM livre WHERE livre_id = 6;     -- 1

UPDATE livre SET prix = ROUND(prix * 1.05, 2);     -- UPDATE 8
```

Le compte préalable annonçait une ligne, le compte rendu en déclare huit : le `WHERE` a été oublié, et tous les prix de la bibliothèque viennent d'augmenter. Aucune erreur, aucune alerte. Si la commande tournait dans une transaction, un `ROLLBACK` la rattrape (section 8) ; sinon, il reste la sauvegarde ou, sur Snowflake, le voyage dans le temps (Cours 11).

Piège — Un `UPDATE` ou un `DELETE` sans `WHERE` touche toute la table. La parade tient en trois gestes : écrire d'abord le `SELECT COUNT(*)` avec le `WHERE` voulu, transformer ce même `SELECT` en `UPDATE` sans retoucher le `WHERE`, et comparer le compte rendu au compte préalable avant de valider.

`UPDATE … FROM` modifie une table à partir d'une autre, comme une jointure. Le fournisseur envoie une liste de prix, rangée dans une table `tarif` :

isbn	nouveau_prix
978-2-9800001-0-5	26,95
978-2-9800001-0-5	25,95
0-9690001-2-X	19,95

```
UPDATE livre AS l
SET prix = t.nouveau_prix
FROM tarif AS t
WHERE t.isbn = l.isbn;
```

`UPDATE 2`, et aucune erreur, alors que la liste contient deux prix pour le même livre. Le livre 1 a reçu l'un des deux, 26,95 $ lors de notre essai ; rien ne garantit que ce sera le même la prochaine fois. DuckDB accepte la même commande tout aussi silencieusement, et son compte rendu compte les correspondances de la source, deux pour un seul livre. Avant une mise à jour par jointure, vérifiez que la clé de jointure est unique dans la source, avec la requête de la section 3.

`DELETE` retire des lignes, avec la même prudence sur le `WHERE` :

```
DELETE FROM emprunt
WHERE rendu_le < '2026-03-01';
```

`DELETE 3` : les emprunts 1, 2 et 3, rendus avant mars. Les trois emprunts en cours restent, parce qu'une comparaison avec leur `rendu_le` à `NULL` n'est pas vraie (Cours 01, section 8). Et la clé étrangère de la section 3 refuserait qu'on supprime un membre qui a encore des emprunts.

© 2026 iAlexMG. Tous droits réservés.

Pour vider une table entière, `TRUNCATE` fait le travail d'un `DELETE` sans `WHERE`, autrement :

	`DELETE FROM t`	`TRUNCATE t`
Choisir les lignes	oui, par un `WHERE`	non, toute la table part
Vitesse sur une grosse table	lente, ligne par ligne	immédiate, la table est vidée d'un bloc
Table visée par une clé étrangère	refuse les lignes encore référencées	refuse la commande entière
Compteur d'identité	intact	intact, ou remis à zéro avec `RESTART IDENTITY` sur PostgreSQL
Annulable dans une transaction, sur PostgreSQL et DuckDB	oui	oui

Sur la bibliothèque, `TRUNCATE membre` échoue : « ne peut pas tronquer une table référencée dans une contrainte de clé étrangère ». Le Cours 01 range `TRUNCATE` dans le DDL, et c'en est un proche parent : il agit sur le contenant plutôt que ligne par ligne.

Portabilité — `UPDATE … FROM` est une extension, que nos deux terrains partagent, mesuré, et que Snowflake écrit de la même façon d'après sa documentation, où le doublon de source reste muet tant que le paramètre `ERROR_ON_NONDETERMINISTIC_UPDATE` garde sa valeur par défaut. Les deux écritures qui font le même travail sans cette clause sont le `MERGE` de la section suivante et la sous-requête corrélée dans le `SET` : c'est par l'une des deux qu'on passe sur un moteur qui ne l'a pas. Pour supprimer à partir d'une autre table, nos deux terrains écrivent `DELETE FROM … USING …`, mesuré.

6. `MERGE` et l'upsert

Labo : `Labo_03_OLTP_SQL.md` et `Labo_03_OLAP_SQL.md`, partie 6.

Un chargement bute souvent sur la même alternative : si la ligne existe déjà, on la met à jour ; sinon, on l'insère. Cette opération double porte un nom, l'**upsert**, et `MERGE` l'écrit en une commande. Un arrivage apporte un nouveau prix pour un livre connu et un livre nouveau, rangés dans une table `arrivage` qui a les mêmes colonnes que `livre` :

```
MERGE INTO livre AS l
USING arrivage AS a
   ON a.isbn = l.isbn
WHEN MATCHED THEN
    UPDATE SET prix = a.prix
WHEN NOT MATCHED THEN
    INSERT (livre_id, isbn, titre, auteur, genre, annee_parution, nb_pages, prix)
    VALUES (a.livre_id, a.isbn, a.titre, a.auteur, a.genre, a.annee_parution, a.nb_pages, a.prix)
RETURNING merge_action(), l.livre_id, l.titre, l.prix;
```


© 2026 iAlexMG. Tous droits réservés.

merge_action	livre_id	titre	prix
UPDATE	6	Après la guerre	17,95
INSERT	9	La Ruelle des Érables	26,95

La cible, `livre`, suit `MERGE INTO` ; la source suit `USING` ; `ON` dit comment les apparier. Chaque ligne de la source tombe dans l'une des branches : `WHEN MATCHED` quand elle trouve sa ligne dans la cible, `WHEN NOT MATCHED` sinon. Une branche peut porter une condition supplémentaire, `WHEN MATCHED AND a.prix IS NULL THEN DELETE`, et le `RETURNING merge_action()` de PostgreSQL dit ce qui est arrivé à chaque ligne. DuckDB écrit la même commande, et `merge_action` sans parenthèses.

Une troisième branche vise l'autre côté : `WHEN NOT MATCHED BY SOURCE` s'applique aux lignes de la cible que la source ne mentionne pas. C'est l'outil d'une synchronisation complète, où ce que la source a retiré doit disparaître de la cible :

```
MERGE INTO stock AS s
USING inventaire AS i
   ON i.isbn = s.isbn
WHEN MATCHED THEN UPDATE SET qte = i.qte
WHEN NOT MATCHED THEN INSERT (isbn, qte) VALUES (i.isbn, i.qte)
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

Mesuré sur un `stock` de trois livres, A, B et C, et un `inventaire` qui ne compte que A et D : A est mis à jour, D inséré, B et C supprimés, et `stock` ne garde que A et D, à l'identique sur nos deux terrains. La branche est puissante et dangereuse à la fois : une source livrée incomplète efface de la cible tout ce qui lui manque. `RETURNING` sur un `MERGE`, `merge_action()` et `WHEN NOT MATCHED BY SOURCE` sont arrivés avec PostgreSQL 17, d'après ses notes de version ; une version plus ancienne ne les connaît pas.

Relancez la même commande : les deux lignes tombent maintenant dans `WHEN MATCHED`, et rien n'est inséré deux fois. C'est la propriété qui fait de `MERGE` l'outil des chargements rejouables, que le Cours 11 approfondit.

Le piège est celui de l'`UPDATE … FROM`, et la conclusion dépend du moteur. Appliqué à la table `tarif`, où un livre a deux prix, `MERGE` refuse de travailler sur PostgreSQL :

```
MERGE INTO livre AS l
USING tarif AS t
   ON t.isbn = l.isbn
WHEN MATCHED THEN
    UPDATE SET prix = t.nouveau_prix;
```

PostgreSQL répond « MERGE command cannot affect row a second time », en anglais même sur un serveur français, et n'écrit rien. Le même défaut de la source passe en silence dans un `UPDATE … FROM` et bloque le `MERGE`. DuckDB, lui, l'accepte sans erreur, mesuré en version 1.5.5 : il applique les deux prix l'un sur l'autre, et le livre garde l'un des deux, comme avec l'`UPDATE … FROM`. Le garde-fou n'est donc pas dans `MERGE` mais dans le moteur, et il disparaît quand le chargement change de moteur.

© 2026 iAlexMG. Tous droits réservés.

Portabilité — Snowflake refuse la source en double, d'après sa documentation, tant que son paramètre `ERROR_ON_NONDETERMINISTIC_MERGE` garde sa valeur par défaut.

PostgreSQL offre une seconde écriture de l'upsert, plus courte quand la source tient en quelques valeurs, et DuckDB l'a reprise : l'`INSERT` qui sait quoi faire en cas de conflit.

```sql
INSERT INTO livre (livre_id, isbn, titre, prix)
VALUES (6, '978-2-9800001-6-7', 'Après la guerre', 18.95)
ON CONFLICT (isbn) DO UPDATE SET prix = EXCLUDED.prix;
```

`EXCLUDED` désigne la ligne qu'on tentait d'insérer. La colonne du conflit doit porter une contrainte `UNIQUE` ou une clé primaire, sans quoi le moteur ne saurait pas reconnaître le conflit. La variante `ON CONFLICT DO NOTHING` ignore simplement les lignes déjà présentes.

À retenir — `MERGE` veut une source où chaque clé n'apparaît qu'une fois, et tous les moteurs ne le vérifient pas pour vous. Vérifiez-le avant de charger, ou dédoublonnez la source dans la requête qui la produit.

Portabilité — `MERGE` et `ON CONFLICT` existent sur nos deux terrains. PostgreSQL a `MERGE` depuis sa version 15, et son `RETURNING` et sa branche `WHEN NOT MATCHED BY SOURCE` depuis la 17 ; DuckDB 1.5.5 a les trois, mesuré, et écrit `merge_action` sans parenthèses. Snowflake a `MERGE`, d'après sa documentation. D'un moteur à l'autre, l'upsert change souvent de forme et de nom : cherchez d'abord `MERGE` dans la documentation du vôtre, puis ce qui lui manque.

7. Vues, `ALTER` et `DROP`

Labo : `Labo_03_OLTP_SQL.md` et `Labo_03_OLAP_SQL.md`, partie 7.

Une **vue** est une requête enregistrée sous un nom. Elle ne stocke aucune donnée : chaque fois qu'on l'interroge, le moteur relance sa requête sur les tables du moment.

```sql
CREATE VIEW emprunt_en_retard AS
SELECT e.emprunt_id, m.prenom, m.nom, l.titre, e.retour_prevu
FROM emprunt AS e
JOIN membre  AS m ON m.membre_id = e.membre_id
JOIN livre   AS l ON l.livre_id = e.livre_id
WHERE e.rendu_le IS NULL
  AND e.retour_prevu < DATE '2026-05-20';

SELECT * FROM emprunt_en_retard;
```

Deux lignes, Marc Côté et Émile Gagnon, les emprunts 5 et 8. Marc rapporte son livre, un `UPDATE` enregistre le retour, et la vue n'en montre plus qu'une, sans qu'on l'ait touchée. C'est tout l'intérêt d'une vue : la logique s'écrit une fois, et tout le monde lit la même définition du retard. C'est aussi sa limite : elle suit les données, y compris les mauvaises. Un `UPDATE emprunt SET rendu_le = …` sans `WHERE` la viderait d'un coup, et elle annoncerait fièrement qu'aucun livre n'est en retard.

© 2026 iAlexMG. Tous droits réservés.

`CREATE OR REPLACE VIEW` remplace la définition d'une vue sans la supprimer. Une vue qui doit, au contraire, garder un résultat calculé s'appelle une vue matérialisée ; elle se rafraîchit à la demande et relève de l'optimisation, hors du périmètre de ce cours.

`ALTER TABLE` modifie une table qui existe. Les formes courantes :

Besoin	Commande
ajouter une colonne	`ALTER TABLE livre ADD COLUMN langue VARCHAR(2) NOT NULL DEFAULT 'fr'`
renommer une colonne	`ALTER TABLE livre RENAME COLUMN nb_pages TO pages`
changer un type	`ALTER TABLE livre ALTER COLUMN nb_pages TYPE INTEGER USING …`
ajouter une contrainte	`ALTER TABLE livre ADD CONSTRAINT … CHECK (…)` (section 3)
supprimer une colonne	`ALTER TABLE livre DROP COLUMN langue`

La première remplit d'emblée les huit livres existants avec « fr ». La troisième a besoin de dire comment convertir chaque valeur, et c'est là que la colonne `nb_pages` se corrige enfin :

```sql
ALTER TABLE livre
ALTER COLUMN nb_pages TYPE INTEGER
USING CASE WHEN nb_pages ~ '^[0-9]+$' THEN CAST(nb_pages AS INTEGER) END;
```

Écrit avec un simple `CAST`, le changement échoue sur « inconnu » (Cours 02, section 4) ; avec le `CASE`, le livre 5 reçoit `NULL` et les sept autres leur nombre de pages.

Une structure a des dépendants, et c'est ce qui rend `ALTER` et `DROP` délicats. Deux comportements de PostgreSQL méritent d'être connus. Une vue écrite avec `SELECT *` fige sa liste de colonnes à sa création : après l'ajout de `langue`, `SELECT * FROM` la vue ne montre toujours pas la nouvelle colonne. Et un changement de type est refusé tant qu'une vue lit la colonne : « ne peut pas modifier le type d'une colonne utilisée dans une vue ou une règle ». DuckDB fait l'inverse sur les deux points : il garde le texte de la vue et relit l'étoile à chaque interrogation, si bien que la colonne ajoutée y paraît, et il laisse changer le type d'une colonne lue par une vue, qui suit. Deux moteurs, deux comportements pour la même vue : c'est la meilleure raison de nommer ses colonnes.

`DROP TABLE` supprime la table, ses données et sa structure. PostgreSQL refuse de supprimer une table dont d'autres objets dépendent :

```sql
DROP TABLE livre;
-- n'a pas pu supprimer table livre car d'autres objets en dépendent
-- DÉTAIL : contrainte emprunt_livre_id_fkey sur table emprunt dépend de table livre
--          vue emprunt_en_retard dépend de table livre
```

`DROP TABLE livre CASCADE` passe outre, et le détail de ce qu'il emporte mérite d'être lu : la vue disparaît, la clé étrangère d'`emprunt` aussi, mais la table `emprunt` et ses douze lignes restent, désormais libres de citer des livres qui n'existent plus. DuckDB, lui, supprime la table sans consulter les vues, qui échoueront

© 2026 iAlexMG. Tous droits réservés.

à leur prochaine interrogation ; il ne refuse la suppression que si une clé étrangère vise la table. `DROP TABLE IF EXISTS` évite l'erreur quand la table est déjà absente, ce qui rend un script de nettoyage rejouable.

À retenir — Dans une vue, nommez les colonnes plutôt que d'écrire `SELECT *`. Et n'écrivez jamais `CASCADE` sans avoir lu la liste de ce qu'il va emporter : la commande sans `CASCADE` vous la donne, sous forme d'erreur.

8. Ce qui s'annule et ce qui ne s'annule pas

Labo : `Labo_03_OLTP_SQL.md` et `Labo_03_OLAP_SQL.md`, partie 8.

Par défaut, psql, pgAdmin, DBeaver et la ligne de commande DuckDB valident chaque commande dès qu'elle réussit : c'est le mode d'autovalidation. Pour pouvoir revenir en arrière, on ouvre une transaction avec `BEGIN`, puis on la termine par `COMMIT`, qui valide tout, ou par `ROLLBACK`, qui annule tout. Le Cours 11 est consacré aux transactions ; ce cours n'en retient que ce qu'il faut pour écrire sans casse.

Pour le DML, tous les moteurs de ce cours se comportent pareil : un `INSERT`, un `UPDATE`, un `DELETE` ou un `MERGE` faits dans une transaction s'annulent jusqu'au `COMMIT`. Pour le DDL, ils divergent, comme l'annonçait le Cours 01 (section 2), et voici l'essai qui le montre :

```sql
BEGIN;
UPDATE livre SET prix = 19.95 WHERE livre_id = 6;
CREATE TABLE essai_ddl (x INTEGER);
ROLLBACK;
```

Sur PostgreSQL, le `ROLLBACK` annule les deux commandes : le livre 6 garde son prix de 16,95 $, et la table `essai_ddl` n'a jamais existé. PostgreSQL sait annuler un `CREATE`, un `ALTER`, un `DROP` et un `TRUNCATE`, et DuckDB aussi : le labo OLAP y annule un `CREATE TABLE` et un `DROP TABLE` par `ROLLBACK`. D'après sa documentation, Snowflake fait autrement : un DDL y valide d'office la transaction en cours, puis s'exécute à part. Le même bloc y laisserait la hausse de prix validée et la table créée, et le `ROLLBACK` n'aurait plus rien à annuler.

Commande	PostgreSQL et DuckDB	Snowflake, d'après sa documentation
`INSERT`, `UPDATE`, `DELETE`, `MERGE` dans une transaction	annulables jusqu'au `COMMIT`	annulables jusqu'au `COMMIT`
`CREATE`, `ALTER`, `DROP` dans une transaction	annulables	validés d'office, avec tout ce qui précède dans la transaction
n'importe quelle commande hors transaction	validée dès qu'elle réussit	validée dès qu'elle réussit

Le « validé d'office » n'est pas propre à Snowflake : Oracle et MySQL valident eux aussi d'office avant et après une commande de structure, d'après leur documentation, et le Cours 11 y revient. Nos deux terrains font donc exception, et c'est ce qui est rare. La conséquence pratique est considérable : sur les deux terrains de la formation, une migration de structure entière peut réussir ou échouer d'un bloc ; sur Snowflake, on sépare le DDL du DML et on prévoit de quoi réparer. Snowflake compense par le voyage dans le temps, qui permet de relire une table telle qu'elle était, ou de récupérer une table supprimée par `UNDROP TABLE` (Cours 11). DuckDB n'a pas ce filet : une table supprimée hors transaction est perdue, sauf copie du fichier de l'entrepôt.

© 2026 iAlexMG. Tous droits réservés.

À retenir — Avant une écriture qui compte : `BEGIN`, la commande, un `SELECT` qui vérifie le résultat, puis `COMMIT` ou `ROLLBACK`. Sur Snowflake, ne glissez jamais un DDL au milieu : il validerait ce que vous vouliez encore pouvoir annuler.

9. Sur les deux terrains : OLTP et OLAP

Labo : `Labo_03_OLTP_SQL.md`, partie 9.

La syntaxe de ce cours est presque la même sur les deux moteurs. Ce qui change, ce sont les garanties que le moteur offre derrière la syntaxe.

Ce qu'on veut	**Voie OLTP (PostgreSQL)**	**Voie OLAP (DuckDB)**
Un bac à sable	un schéma, `labo03`	un préfixe, `LABO03_`, dans `dbo`
Recréer une table d'un coup	`DROP TABLE IF EXISTS`, puis `CREATE TABLE`	`CREATE OR REPLACE TABLE`
Une clé générée	`GENERATED … AS IDENTITY`, lue par `RETURNING`	une séquence appelée par la valeur par défaut, lue par `RETURNING`
Un texte plus long que son `VARCHAR(n)`	refusé	accepté en entier
Un doublon de clé primaire	refusé	refusé
Une contrainte `CHECK`	appliquée	appliquée
Un `INSERT` sans liste, trop court	complété par les valeurs par défaut	refusé
`UPDATE … FROM` sur une source en double	accepté, une valeur au hasard	accepté, une valeur au hasard
`MERGE` sur une source en double	refusé	accepté, une valeur au hasard
L'upsert	`MERGE` ou `INSERT … ON CONFLICT`	`MERGE` ou `INSERT … ON CONFLICT`
Une vue `SELECT *` après un ajout de colonne	figée sur les colonnes d'origine	elle suit la table
Supprimer une table lue par une vue	refusé sans `CASCADE`	accepté, la vue casse ensuite
Un `CREATE TABLE` dans une transaction	annulé par le `ROLLBACK`	annulé par le `ROLLBACK`

Deux leçons débordent le tableau.

La première, c'est que les garanties ne sont pas dans le langage mais dans le moteur. La même déclaration `VARCHAR(3)` est une limite sur le transactionnel et un commentaire sur l'entrepôt de la formation ; le même `MERGE` protège de la source en double d'un côté et pas de l'autre. Et sur un entrepôt infonuagique comme Snowflake, d'après sa documentation, la même `PRIMARY KEY` n'est plus qu'une étiquette. Là où le moteur ne protège pas, la protection se déplace vers le chargement et vers des tests qu'il faut écrire soi-même.

© 2026 iAlexMG. Tous droits réservés.

La seconde, c'est que les deux mondes n'écrivent pas de la même façon. Le transactionnel vit d'écritures unitaires, une commande, un client, un emprunt, chacune vérifiée par ses contraintes. L'entrepôt écrit en masse, par `INSERT … SELECT`, `CREATE TABLE … AS` et `MERGE`, des milliers de lignes à la fois, et c'est la qualité de la requête source qui fait la qualité de la table.

Le même travail, deux fois — Le Cours 03 a deux laboratoires, `Labo_03_OLAP_SQL.md` et `Labo_03_OLTP_SQL.md`, qui font les mêmes gestes d'écriture sur les deux terrains. Faites l'OLAP d'abord : le labo OLTP se termine sur le tableau des écarts, qui n'a de sens qu'une fois les deux faits.

Points clés à retenir

- On écrit dans un bac à sable, schéma ou préfixe, jamais là où les autres lisent.
- Le type d'une colonne refuse certaines valeurs et en arrondit d'autres sans prévenir : `NUMERIC(6,2)` range 12,345 en 12,35.
- `CREATE TABLE … AS` copie les noms et les types, jamais les contraintes, les valeurs par défaut ni l'identité.
- Cinq contraintes : `NOT NULL`, `PRIMARY KEY`, `UNIQUE`, clé étrangère, `CHECK`. `UNIQUE` et `CHECK` laissent passer `NULL`. PostgreSQL et DuckDB les appliquent toutes ; un entrepôt infonuagique comme Snowflake, d'après sa documentation, n'applique que `NOT NULL` et `CHECK` sur une table ordinaire. DuckDB ne vérifie pas la longueur d'un `VARCHAR(n)`.
- Une valeur par défaut ne s'applique qu'à une colonne omise, pas à un `NULL` écrit.
- Nommez toujours les colonnes d'un `INSERT` : sans liste, les valeurs vont aux colonnes par position, et une inversion passe en silence.
- Après un chargement à numéros explicites, remettez le compteur d'identité au-delà du plus grand.
- Comptez avant d'écrire, et comparez le compte rendu (`UPDATE 8`) au compte attendu : un `WHERE` oublié ne lève aucune erreur.
- `UPDATE … FROM` accepte une source en double et prend une valeur au hasard ; `MERGE` la refuse sur PostgreSQL, pas sur DuckDB. Vérifiez l'unicité de la clé dans la source avant l'un comme l'autre.
- Une vue suit les données, bonnes ou mauvaises. Nommez ses colonnes, et lisez ce qu'un `CASCADE` emporte avant de l'écrire.
- Le DML s'annule jusqu'au `COMMIT` partout. Le DDL s'annule sur PostgreSQL et DuckDB ; sur Snowflake, il valide d'office la transaction en cours.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_03_SQL.md`. Faites-les après les deux laboratoires : plusieurs énoncés supposent que vous ayez vu de vos yeux le `VARCHAR(3)` qui garde quatre caractères, le compteur d'identité en retard et le `MERGE` qui accepte ou refuse une source en double selon le moteur.

© 2026 iAlexMG. Tous droits réservés.